Tool Learning and Function Calling for LLM Agents: A Comprehensive Survey

Abstract

The integration of Large Language Models (LLMs) with external tools and function calling interfaces has emerged as a pivotal paradigm shift, transforming static language models into dynamic, autonomous agents capable of interacting with the physical and digital world. This survey provides a comprehensive review of the rapidly evolving field of tool learning and function calling, synthesizing insights from 300 recent academic studies. We systematically categorize methodological advancements into core dimensions: architectural frameworks for tool invocation, optimization strategies for selection and planning, training paradigms ranging from prompt engineering to reinforcement learning, and the critical challenges of evaluation, safety, and deployment. Our analysis reveals a distinct evolution from simple single-step API calls to complex, multi-agent orchestration systems capable of hierarchical planning, self-correction, and autonomous tool creation. We examine the trade-offs between parameter-efficient fine-tuning and training-free retrieval mechanisms, the emergence of specialized benchmarks for nested and multi-turn interactions, and the growing security concerns regarding adversarial tool manipulation and resource amplification. By consolidating evidence from diverse domains including scientific discovery, enterprise automation, and edge computing, this paper identifies key bottlenecks in long-horizon planning, context management, and robustness, offering a structured outlook for future research in agentic AI.

1. Introduction

The capabilities of Large Language Models (LLMs) have expanded exponentially, yet their inherent reliance on static parametric knowledge limits their utility in dynamic, real-world applications requiring up-to-date information, precise computation, or interaction with external systems [1]. To bridge this gap, the research community has pivoted towards "Tool Learning," a paradigm where LLMs are augmented with the ability to invoke external functions, APIs, and code executors [2]. This evolution marks a transition from passive text generation to active agency, enabling models to perform complex tasks such as scientific experimentation [3], financial analysis [4], and software development [5].

The foundational concept involves equipping LLMs with a set of tools—defined as external function interfaces executable outside the model—that extend their cognitive and operational boundaries [2]. Early approaches relied heavily on few-shot prompting and in-context learning to guide models in generating structured API calls [6]. However, as tasks grew in complexity, necessitating multi-step reasoning and dependency management, simple prompting proved insufficient due to hallucination, format errors, and planning failures [7]. Consequently, the field has diversified into sophisticated methodologies encompassing retrieval-augmented generation (RAG) for tool selection [8], hierarchical planning architectures [9], and specialized fine-tuning strategies that internalize tool semantics [10].

Recent literature highlights a bifurcation in approach: one stream focuses on optimizing massive, general-purpose models through advanced prompting and retrieval mechanisms to handle vast tool libraries [11], while another explores the efficacy of small language models (SLMs) fine-tuned for specific tool-use domains, often outperforming larger counterparts in latency-constrained environments [12][13]. Furthermore, the scope of tool use has expanded from predefined API sets to autonomous tool creation, where agents synthesize new functions from code repositories or docu-

mentation [5][14].

Despite these advancements, significant challenges remain. Models frequently struggle with long-horizon planning, exhibiting performance degradation as dependency depth increases [15]. Security vulnerabilities, such as prompt injection attacks targeting tool selection [16] and resource amplification via malicious tool chains [17], pose critical risks. Additionally, the lack of standardized evaluation frameworks for multi-turn, stateful interactions has led to fragmented progress [18]. This survey synthesizes 300 relevant papers to provide a unified taxonomy of tool learning methods, critically evaluate current benchmarks, and outline the open problems defining the next frontier of agentic AI.

2. Method

The methodology of tool learning and function calling can be decomposed into several interconnected subsystems: the architectural mechanisms for invoking tools, the strategies for selecting and retrieving appropriate tools from large corpora, the planning algorithms for multi-step execution, the training paradigms for internalizing tool capabilities, and the frameworks for evaluation and safety.

2.1 Architectural Frameworks for Tool Invocation

The fundamental architecture of a tool-using agent dictates how natural language intents are mapped to executable function calls. This mapping has evolved from monolithic generation pipelines to modular, decoupled systems designed to enhance accuracy and reduce latency.

2.1.1 Decoupled Selection and Generation Pipelines A recurring insight in recent literature is that conflating tool selection with parameter generation often leads to suboptimal performance, particularly for smaller models or complex toolsets. To address this, several frameworks propose decoupling these stages. The Natural Language Tools (NLT) approach replaces rigid JSON schemas with natural language YES/NO selection lists, employing a three-stage pipeline where a selector model decides on tool usage before a parser executes the call [19]. Similarly, FREYR decomposes the process into four discrete modules: intent recognition, parameter generation, output summarization, and conversation handling, allowing for heterogeneous model allocation across stages [20]. This modularization reduces input token consumption by isolating tool descriptions to specific inference passes, significantly improving step completion rates compared to monolithic approaches [20].

In enterprise settings, the "Agent-First" API design rethinks service interfaces by decomposing interactions into six semantic verbs (search, resolve, preview, execute, verify, recover), forming a goal-achievement loop that shifts resolution burden from probabilistic LLM reasoning to deterministic backend logic [21]. This contrasts with traditional CRUD-based calls, reducing ambiguity-induced errors by 37.5% in production SaaS environments [21].

2.1.2 Structured Output and Format Enforcement Ensuring syntactic correctness of function calls is paramount. Methods like FANTASE integrate State-tracked Constrained Decoding (SCD) using a Constrained Token Search Trie (CTST) to dynamically enforce API documentation constraints during generation, guaranteeing faithfulness to schemas [22]. Alternatively, Hammer employs function masking, replacing function and parameter names with random strings during training to force the model to rely on semantic descriptions rather than naming biases, thereby improving cross-domain generalization [23]. For on-device applications, Octopus v2 maps discrete

API functions to unique special tokens within the tokenizer vocabulary, transforming function selection into a single-token classification problem that eliminates retrieval overhead and achieves 99.5% accuracy on Android APIs [24].

2.1.3 Streaming and Concurrent Execution Latency remains a bottleneck in sequential tool calling. GhostShell introduces streaming function calls for embodied programming, parsing LLM output streams incrementally via XML tokens to initiate actions before the full response is generated [25]. On the system level, AsyncLM decouples token generation from function execution using a Context Markup Language (CML), allowing parallel background processing and achieving a 5.4x latency speedup over synchronous methods [26]. Furthermore, LLMOrch constructs Function-call Relation Graphs (FRG) to identify data dependencies and mutual-exclusion constraints, enabling fine-grained parallel execution of independent tool calls and reducing latency by up to 2.18x [27].

2.2 Tool Retrieval and Selection Strategies

As the number of available tools scales into the thousands, retrieving the correct tool becomes a critical challenge. Naive semantic similarity often fails to capture functional utility, leading to the development of behavior-aligned and execution-grounded retrieval methods.

2.2.1 Semantic and Behavior-Aligned Retrieval Standard dense retrieval based on query-tool description similarity is often insufficient. The Behavior-Aligned Retriever (BAR) employs contrastive learning to construct positive pairs that match both semantic proximity and invocation behavior (call/no-call), improving direct response rates by 8.5% over baseline retrievers [28]. ToolScope advances this by constructing an undirected pruning graph where nodes represent tools and edges denote semantic equivalence, merging redundant tools to improve embedding space distribution and retrieval accuracy [29]. For massive tool libraries exceeding context limits, AnyTool implements a three-tier hierarchical retriever (meta-agents, category agents, tool agents) to navigate 16,000+ APIs via divide-and-conquer, significantly outperforming flat retrieval baselines [9].

2.2.2 Execution-Grounded Validation Textual relevance does not guarantee functional correctness. GRETEL integrates real-world API execution trials directly into the retrieval re-ranking phase, employing a Plan-Execute-Evaluate cycle to validate candidate tools before selection [30]. This execution-grounded evidence provides a superior signal for tool selection compared to textual similarity alone. Similarly, Causal Minimal Tool Filtering (CMTF) constructs precondition-effect dependency graphs to expose only the immediate executable frontier of tools, reducing "ToolChoice-Confusion" caused by semantically plausible but causally premature options [31].

2.2.3 Dynamic and Context-Aware Selection Dynamic selection mechanisms adapt to the evolving state of a task. EcoAct initializes agents with a single meta-tool ("tool register") and dynamically fetches full tool descriptions only when deemed necessary, reducing token costs by 54% while maintaining pass rates [32]. In multi-agent systems, Tool-to-Agent Retrieval embeds tools and parent agents in a shared vector space linked by ownership metadata, preventing context dilution and improving recall by 19.4% [33]. ScaleMCP treats Model Context Protocol (MCP) servers as a single source of truth, using hash-based auto-synchronization to ensure real-time consistency between tool definitions and agent knowledge [34].

2.3 Planning and Multi-Step Orchestration

Single-step function calling has matured; the current frontier lies in multi-step, multi-hop, and nested tool orchestration. This requires sophisticated planning algorithms capable of handling dependencies, backtracking, and state tracking.

2.3.1 Graph-Based and Hierarchical Planning Graph structures are increasingly used to model tool dependencies. GRAFT (Graph-Tokenized LLMs) internalizes directed tool dependencies by mapping each tool node to a special token and training the model to recover valid successor edges, enabling direct dependency-aware generation without external search [35]. For complex workflows, Flash-Searcher decomposes tasks into subtasks forming a Directed Acyclic Graph (DAG), executing independent reasoning paths concurrently to improve efficiency [36]. Hierarchical approaches like AnyTool and ToolLLM employ Depth-First Search-based Decision Trees (DFSDT) to explore multiple reasoning traces and retract faulty steps, significantly outperforming linear chains [8][9].

2.3.2 Backward and Reverse Chaining Forward planning often deviates from final goals. Reverse Chain executes backward reasoning, starting from the target API and iteratively resolving arguments via recursive sub-API calls, achieving 92% accuracy on compositional tasks by anchoring the plan to the final objective [37]. Similarly, SoAy constructs an API dependency graph to enumerate valid calling sequences (solutions) before generating code, resolving parameter dependencies more effectively than implicit methods [38].

2.3.3 State Management and Memory Maintaining state across long horizons is critical. Sum2Act operates via iterative cycles of action proposal and summarization, where a State Manager synthesizes API observations into compact representations, explicitly recording failure reasons to maintain accurate task states [39]. VehicleWorld bypasses sequential API discovery by ingesting full JSON system states and predicting target states directly, generating minimal code to modify object attributes and reducing interaction rounds [40]. For computer-use agents, OSExpert employs a GUI-DFS algorithm to systematically explore digital environments, consolidating discovered unit functions into a skill library for reuse [41].

2.4 Training Paradigms for Tool Learning

While prompting remains popular, fine-tuning and reinforcement learning (RL) are essential for internalizing tool syntax and logic, especially for smaller models or specialized domains.

2.4.1 Supervised Fine-Tuning (SFT) and Data Synthesis High-quality synthetic data is crucial for SFT. ToolACE employs a "speciation-adaptation-evolution" process to generate diverse APIs and uses model loss as a proxy for data complexity to dynamically adjust query difficulty [42]. TOOLFLOW constructs a tool graph linking APIs via parameter/return-value similarity, executing random walks to sample correlated tool subsets for coherent dialogue synthesis [43]. For domain-specific applications, BioTool utilizes reverse query generation from executed API observations to ensure data grounding, enabling small models to outperform large proprietary ones in biomedical tool calling [44].

2.4.2 Reinforcement Learning and Preference Optimization RL is increasingly used to optimize tool usage policies. StepTool models tool learning as a Markov Decision Process, implementing step-grained reward shaping that balances invocation success and task contribution, out-

performing final-only reward baselines [45]. FunReason integrates a Self-Refinement Multiscale Loss (SRML) that dynamically balances reasoning and function call accuracy, outperforming GPT-4o on the BFCL benchmark [46]. Direct Preference Optimization (DPO) is also effective; DiaTool-DPO constructs paired datasets of chosen and rejected dialogue trajectories to learn context-dependent dialogue flow, improving slot-filling accuracy by 44% over SFT-only baselines [47].

2.4.3 Parameter-Efficient and Small Model Adaptation Small Language Models (SLMs) can excel at tool use with targeted fine-tuning. A study on OPT-350M demonstrated that a 350M parameter model fine-tuned on ToolBench could surpass ChatGPT-CoT by 51% in pass rate, validating that task-specific optimization outperforms scale-based approaches for structured manipulation [48]. Octopus fine-tunes small models (2B-7B) using curriculum learning with hard negative samples, achieving higher accuracy than GPT-4 on specific function calling tasks [13]. LoKI further mitigates catastrophic forgetting by identifying low-contribution neurons in FFN layers for knowledge implanting, preserving general capabilities while acquiring tool skills [49].

2.5 Autonomous Tool Creation and Self-Improvement

Beyond using predefined tools, advanced agents can create, refine, and optimize their own toolsets.

2.5.1 Tool Synthesis and Refinement WALT (Web Agents that Learn Tools) reverse-engineers latent website functionality into validated callable tools with input schemas, shifting the computational burden from fragile reasoning to reliable execution [50]. SkillWeaver autonomously converts interaction trajectories into executable Python API libraries, enabling efficient skill transfer across agents and websites [14]. TOOLMAKER executes a two-stage workflow to clone code repositories, resolve dependencies, and generate reproducible Docker images, allowing agents to bypass building complex tools from scratch [5].

2.5.2 Self-Evolution and Meta-Learning Self-evolving agents continuously improve their capabilities. STELLA orchestrates a Manager-Dev-Critic-ToolCreation loop where a dedicated agent autonomously builds and integrates new bioinformatics tools, doubling performance on biomedical exams through test-time evolution [51]. MAGE externalizes self-knowledge into a co-evolutionary knowledge graph, allowing frozen models to improve via retrieval of failure corrections and success traces [52]. HarnessForge formulates agent adaptation as a co-evolution of harness structure and policy behavior, using fault-guided tailoring to ensure executable compatibility [53].

2.6 Evaluation Benchmarks and Metrics

The proliferation of tools has necessitated diverse benchmarks to evaluate different aspects of agent capability.

2.6.1 Multi-Step and Nested Evaluation NESTOOLS evaluates nested tool learning abilities by mapping Python function nesting patterns to API sequences, revealing that performance degrades significantly with increased nesting depth [54]. ToolHop constructs query-driven data with strict interdependencies between tools, providing locally executable code for objective evaluation of multi-hop reasoning [55]. FuncBenchGen generates contamination-free synthetic benchmarks via DAG traversal, controlling task complexity through structural parameters to prevent data leakage [15].

2.6.2 Long-Context and Multilingual Robustness LongFuncEval assesses tool selection accuracy under large catalog sizes and positional variance, finding that LLMs exhibit severe performance drops (up to 91%) as tool response lengths increase [56]. MLCL extends the BFCL benchmark to evaluate multilingual robustness, identifying that models frequently generate semantically correct but operationally invalid calls by copying non-English tokens into parameter values [57].

2.6.3 Real-World and Domain-Specific Benchmarks CallNavi evaluates LLMs on unfiltered API selection from 100+ candidates, introducing an Election Stability Score to quantify output consistency [11]. For scientific domains, ComplexMCP evaluates agents in dynamic, interdependent tool sandboxes with stateful environments, revealing that top-tier LLMs fail to exceed 60% success rates due to retrieval saturation [58]. AutomationBench tests cross-application workflow orchestration via REST APIs, showing that frontier models score below 10% due to failures in persistent data searching [59].

2.7 Safety, Security, and Reliability

As agents gain autonomy, they become susceptible to novel attack vectors and reliability issues.

2.7.1 Adversarial Attacks and Jailbreaking ToolTweak demonstrates that iterative refinement of tool metadata (names and descriptions) can significantly bias agent selection behavior, increasing target selection rates from 20% to 81% [60]. iMIST exploits function calling interfaces to disguise malicious queries as structured parameter filling, bypassing safety filters with high success rates [61]. Sequential Tool Attack Chaining (STAC) distributes malicious objectives over multiple turns with individually harmless tool calls, bypassing standard guardrails with >90% success [62].

2.7.2 Resource Amplification and DoS Beyond Max Tokens reveals stealthy resource amplification attacks where benign MCP servers are manipulated to induce multi-turn tool calling chains, amplifying token usage by 658x and causing Denial-of-Service [17]. ChainFuzzer identifies that 82.74% of vulnerabilities in agent apps require multi-tool execution, highlighting the need for workflow-level security modeling [63].

2.7.3 Reliability and Self-Healing Memory-Induced Tool-Drift formalizes a failure mode where biased personal memories function as implicit steering vectors, pushing activations along latent directions and causing parameter deviations in professional tasks [64]. To counteract failures, a Self-Healing Framework integrates continuous reliability assessment with automated recovery mechanisms, enabling real-time adaptation to hallucinations and execution errors [65]. AgentGuard implements a tripartite inspection mechanism (rules, LLMs, human review) to intercept and validate tool invocations, decoupling security control from agent logic [66].

3. Challenges and Outlook

Despite significant progress, the field of tool learning and function calling faces several persistent challenges that hinder widespread deployment and robustness.

3.1 The Long-Horizon Planning Bottleneck

A consistent finding across multiple studies is the sharp degradation in performance as task complexity and dependency depth increase. Models struggle to maintain coherent plans over long horizons,

often failing to backtrack from erroneous paths or losing track of intermediate states [15][67]. While methods like DFSDT and hierarchical planning mitigate this to some extent, they incur substantial computational costs and latency [8][9]. Future research must focus on developing more efficient search algorithms and memory architectures that can sustain coherence over extended interaction sequences without exponential resource growth.

3.2 Evaluation Fragmentation and Contamination

The landscape of benchmarks is highly fragmented, with many relying on synthetic data that may not reflect real-world API variability or latency [68][15]. Data contamination is a growing concern, as models trained on public benchmarks may overfit to specific evaluation sets rather than generalizing to novel tools [15]. There is a critical need for contamination-free, dynamic evaluation frameworks that utilize live APIs and stateful environments to provide a true measure of agent capability [58][59].

3.3 Security and Alignment Gaps

The integration of tools introduces new attack surfaces. Function arguments exhibit significant alignment discrepancies compared to chat responses, allowing high-success jailbreaks when users coerce models into tool execution modes [69]. Furthermore, the "Tool Illusion" suggests that LLM-synthesized tools may function as one-way capability distillation, benefiting only weaker user models while introducing complexity overhead [70]. Robust defense mechanisms, such as runtime monitoring for Time-of-Check to Time-of-Use (TOCTOU) vulnerabilities and formal verification of tool contracts, are essential for safe deployment [71][21].

3.4 Efficiency and Edge Deployment

Deploying tool-using agents on edge devices remains challenging due to memory and latency constraints. While small models fine-tuned for specific domains show promise [13][48], they often lack the generalization capacity of larger models. Techniques like speculative tool calling [72] and model quantization [73] offer pathways to efficiency, but a unified framework for balancing accuracy, latency, and energy consumption in resource-constrained environments is still lacking.

3.5 Future Directions

Future research should prioritize the development of **universal tool representations** that allow agents to seamlessly interact with heterogeneous APIs without extensive retraining [74]. **Self-evolving agents** that can autonomously discover, validate, and synthesize new tools will be crucial for adapting to dynamic environments [51][14]. Additionally, **human-in-the-loop frameworks** that leverage higher-order cognitive models for strategic teaching could accelerate agent learning and alignment [75]. Finally, establishing **standardized security protocols** for tool invocation and inter-agent communication will be paramount as agentic systems become more pervasive in critical infrastructure.

4. Conclusion

Tool learning and function calling have fundamentally transformed Large Language Models from static knowledge repositories into dynamic, autonomous agents capable of interacting with the world. This survey has synthesized a vast body of literature, highlighting the evolution from simple

prompting to sophisticated architectures involving hierarchical planning, reinforcement learning, and autonomous tool creation. We have identified key methodological families, including decoupled selection pipelines, execution-grounded retrieval, and graph-based planning, while also exposing critical vulnerabilities in security and long-horizon reliability.

The evidence suggests that while current models exhibit impressive capabilities in controlled settings, significant gaps remain in robustness, efficiency, and generalization to open-world scenarios. The path forward requires a concerted effort to develop standardized, contamination-free benchmarks, robust security frameworks, and efficient architectures capable of sustaining long-horizon reasoning. As the field matures, the integration of tool learning with self-evolution and multi-agent collaboration promises to unlock new frontiers in scientific discovery, enterprise automation, and personalized assistance, ultimately realizing the vision of truly autonomous AI systems.

References

[1] Pengyu Zhao, Zijian Jin, Ning Cheng. (2023). An In-depth Survey of Large Language Model-based Artificial Intelligence Agents. arXiv:2309.14365.

[2] Zora Zhiruo Wang, Zhoujun Cheng, Hao Zhu. (2024). What Are Tools Anyway? A Survey from the Language Model Perspective. arXiv:2403.15452.

[3] Daniil A. Boiko, Robert MacKnight, Gabe Gomes. (2023). Emergent autonomous scientific research capabilities of large language models. arXiv:2304.05332.

[4] Siyu An, Qin Li, Junru Lu. (2024). FinVerse: An Autonomous Agent System for Versatile Financial Analysis. arXiv:2406.06379.

[5] Georg Wölflein, Dyke Ferber, Daniel Truhn. (2025). LLM Agents Making Agent Tools. arXiv:2502.11705.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

[7] Nicholas Farn, Richard Shin. (2023). TOOLTALK: EVALUATING TOOL USAGE IN A CONVERSATIONAL SETTING. arXiv:2311.10775.

[8] Yujia Qin, Shihao Liang, Yining Ye. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. arXiv:2307.16789.

[9] Yu Du, Fangyun Wei, Hongyang Zhang. (2024). AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. arXiv:2402.04253.

[10] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal. (2024). Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-task Learning of Granular Tasks. arXiv:2407.00121.

[11] Yewei Song, Xunzhu Tang, Cedric Lothritz. (2025). CallNavi, A Challenge and Empirical Study on LLM Function Calling and Routing. arXiv:2501.05255.

[12] Weizhou Shen, Chenliang Li, Hongzhan Chen. (2024). Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324.

[13] Wei Chen, Zhiyuan Li, Mingyuan Ma. (2024). Octopus: On-device language model for function calling of software APIs. arXiv:2404.01549.

[14] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin. (2025). SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills. arXiv:2504.07079.

[15] Seiji Maekawa, Jackson Hassell, Pouya Pezeshkpour. (2025). TOWARDS RELIABLE BENCHMARKING: A CONTAMINATION FREE, CONTROLLABLE EVALUATION FRAMEWORK FOR MULTI-STEP LLM FUNCTION CALLING. arXiv:2509.26553.

[16] Jiawen Shi, Zenghui Yuan, Guiyao Tie. (2024). Prompt Injection Attack to Tool Selection in LLM Agents. arXiv:2504.19793.

[17] Kaiyu Zhou, Yongsen Zheng, Yicheng He. (2025). Beyond Max Tokens: Stealthy Resource Amplification via Tool Calling Chains in LLM Agents. arXiv:2601.10955.

[18] Jiarui Lu, Thomas Holleis, Yizhe Zhang. (2024). TOOLSANDBOX: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682.

[19] Reid T. Johnson, Michelle D. Pain, Jordan D. West. (2025). Natural Language Tools: A Natural Language Approach to Tool Calling In Large Language Agents. arXiv:2510.14453.

[20] Roberto Gallotta, Antonios Liapis, Georgios N. Yannakakis. (2024). FREYR: A FRAMEWORK FOR RECOGNIZING AND EXECUTING YOUR REQUESTS. arXiv:2501.12423.

[21] Kai Pan, Rong Hou. (2024). Agent-First Tool APIs: Rethinking Enterprise Service Interfaces for LLM-Native Execution. arXiv:2605.10555.

[22] Zhuoer Wang, Leonardo F. R. Ribeiro, Alexandros Papangelis. (2024). FANTAstic SEquences and Where to Find Them: Faithful and Efficient API Call Generation through State-tracked Constrained Decoding and Reranking. arXiv:2407.13945.

[23] Qiqiang Lin, Muning Wen, Qiuying Peng. (2024). Hammer: Robust Function-Calling for On-Device Language Models via Function Masking. arXiv:2410.04587.

[24] Wei Chen, Zhiyuan Li. (2024). Octopus v2: On-device language model for super agent. arXiv:2404.01744.

[25] Jian Gong, Youwei Huang, Bo Yuan. (2024). GhostShell: Streaming LLM Function Calls for Concurrent Embodied Programming. arXiv:2508.05298.

[26] Gim, Seung-seob Lee, Lin Zhong. (2024). ASYNCHRONOUS LLM FUNCTION CALLING. arXiv:2412.07017.

[27] Xiaoxia Liu, Peng Di, Cong Li. (2024). Efficient Function Orchestration for Large Language Models. arXiv:2504.14872.

[28] Yixin Chen, Ying Xiong, Shangyu Wu. (2024). Beyond Semantic Similarity: Reducing Unnecessary API Calls via Behavior-Aligned Retriever. arXiv:2508.14323.

[29] Marianne Menglin Liu, Daniel Garcia, Fjona Parllaku. (2025). ToolScope: Enhancing LLM Agent Tool Use through Tool Merging and Context-Aware Filtering. arXiv:2510.20036.

[30] Zongze Wu, Yani Guo, Churong Liang. (2024). GRETEL: A GOAL-DRIVEN RETRIEVAL AND EXECUTION-BASED TRIAL FRAMEWORK FOR LLM TOOL SELECTION ENHANCING. arXiv:2510.17843.

[31] Rahul Suresh Babu, Laxmipriya Ganesh Iyer. (2024). ToolChoiceConfusion: Causal Minimal Tool Filtering for Reliable LLM Agents. arXiv:2606.06284.

[32] Shaokun Zhang, Jieyu Zhang, Dujian Ding. (2024). EcoAct: Economic Agent Determines When to Register What Action. arXiv:2411.01643.

[33] Elias Lumer, Faheem Nizar, Anmol Gulati. (2024). Tool-to-Agent Retrieval: Bridging Tools and Agents for Scalable LLM Multi-Agent Systems. arXiv:2511.01854.

[34] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah. (2025). ScaleMCP: Dynamic Tool Selection and Auto-Synchronization for LLM Agents via Model Context Protocol. arXiv:2505.06416.

[35] Xinyi Gao, Xinyu Ren, Junliang Yu. (2024). GRAFT: Graph-Tokenized LLMs for Tool Planning. arXiv:2605.11706.

[36] Tianrui Qin, Sinuo Wang, King Zhu. (2025). Flash-Searcher: Fast and Effective Web Agents via DAG-Based Parallel Execution. arXiv:2509.25301.

[37] Yinger Zhang, Hui Cai, Xierui Song. (2024). Reverse Chain: A Generic-Rule for LLMs to Master Multi-API Planning. arXiv:2310.04474.

[38] Yuanchun Wang, Jifan Yu, Zijun Yao. (2025). SoAy: A Solution-based LLM API-using Methodology for Academic Information Seeking. arXiv:2405.15165.

[39] Yulong Liu, Yunlong Yuan, Chunwei Wang. (2024). From Summary to Action: Enhancing Large Language Models for Complex Tasks with Open World APIs. arXiv:2402.18157.

[40] Jie Yang, Jiajun Chen, Zhangyue Yin. (2025). VehicleWorld: A Highly Integrated Multi-Device Environment for Intelligent Vehicle Interaction. arXiv:2509.06736.

[41] Jiateng Liu, Zhenhailong Wang, Rushi Wang. (2025). OSExpert: Computer-Use Agents Learning Professional Skills via Exploration. arXiv:2603.07978.

[42] Weiwen Liu, Xu Huang, Xingshan Zeng. (2024). TOOLACE: WINNING THE POINTS OF LLM FUNCTION CALLING. arXiv:2409.00920.

[43] Zezhong Wang, Xingshan Zeng, Weiwen Liu. (2024). TOOLFLOW: Boosting LLM Tool-Calling Through Natural and Coherent Dialogue Synthesis. arXiv:2410.18447.

[44] Xin Gao, Ruiyi Zhang, Meixi Du. (2025). BioTool: A Comprehensive Tool-Calling Dataset for Enhancing Biomedical Capabilities of Large Language Models. arXiv:2605.05758.

[45] Yuanqing Yu, Zhefan Wang, Weizhi Ma. (2024). StepTool: Enhancing Multi-Step Tool Usage in LLMs through Step-Grained Reinforcement Learning. arXiv:2410.07745.

[46] Bingguang Hao, Maolin Wang, Zengzhuang Xu. (2025). FunReason: Enhancing Large Language Models' Function Calling via Self-Refinement Multiscale Loss and Automated Data Refinement. arXiv:2505.20192.

[47] Sunghee Jung, Donghun Lee, Shinbok Lee. (2024). DiaTool-DPO: Multi-Turn Direct Preference Optimization for Tool-Augmented Large Language Models. arXiv:2504.02882.

[48] Polaris Jhandi, Owais Kazi, Shreyas Subramanian. (2024). Small Language Models for Efficient Agentic Tool Calling: Outperforming Large Models with Targeted Fine-tuning. arXiv:2512.15943.

[49] Runyu Wang, Peng Ping, Zhengyu Guo. (2024). LoKI: Low-damage Knowledge Implanting of Large Language Models. arXiv:2505.22120.

[50] Viraj Prabhu, Yutong Dai, Matthew Fernandez. (2025). WALT: WEB AGENTS THAT LEARN TOOLS. arXiv:2510.01524.

[51] Ruofan Jin, Zaixi Zhang, Mengdi Wang. (2024). STELLA: Self-Evolving LLM Agent for Biomedical Research. arXiv:2507.02004.

[52] Ruiyi Yang, Zechen Li, Hao Xue. (2024). MAGE: Multi-Agent Self-Evolution with Co-Evolutionary Knowledge Graphs. arXiv:2605.10064.

[53] Mingju Chen, Can Lv, Guibin Zhang. (2025). HarnessForge: Joint Harness and Policy Evolution for Adaptive Agent Systems. arXiv:2606.01779.

[54] Han Han, Tong Zhu, Xiang Zhang. (2024). NESTOOLS : A Dataset for Evaluating Nested Tool Learning Abilities of Large Language Models. arXiv:2410.11805.

[55] Junjie Ye, Zhengyin Du, Xuesong Yao. (2024). ToolHop: A Query-Driven Benchmark for Evaluating Large Language Models in Multi-Hop Tool Use. arXiv:2501.02506.

[56] Kiran Kate, Tejaswini Pedapati, Kinjal Basu. (2025). LongFuncEval: Measuring the effectiveness of long context models for function calling. arXiv:2505.10570.

[57] Zheng Luo, T Pranav Kutralingam, Ogochukwu N. Okoani. (2025). Lost in Execution: On the Multilingual Robustness of Tool Calling in Large Language Models. arXiv:2601.05366.

[58] Yuanyang Li, Xue Yang, Longyue Wang. (2025). ComplexMCP: Evaluation of LLM Agents in Dynamic, Interdependent, and Large-Scale Tool Sandbox. arXiv:2605.10787.

[59] Daniel Shepard, Robin Salimans. (2026). AutomationBench: A Benchmark for Cross-Application Workflow Orchestration via REST APIs. arXiv:2604.18934.

[60] Jonathan Sneh, Ruomei Yan, Jialin Yu. (2025). TOOLTWEAK: AN ATTACK ON TOOL SELECTION IN LLM-BASED AGENTS. arXiv:2510.02554.

[61] Zhaoqi Wang, Zijian Zhang, Daqing He. (2024). Jailbreaking Large Language Models through Iterative Tool-Disguised Attacks via Reinforcement Learning. arXiv:2601.05466.

[62] Jing-Jing Li, Jianfeng He, Chao Shang. (2025). STAC: When Innocent Tools Form Dangerous Chains to Jailbreak LLM Agents. arXiv:2509.25624.

[63] Jiangrong Wu, Zitong Yao, Yuhong Nan. (2024). ChainFuzzer: Greybox Fuzzing for Workflow-Level Multi-Tool Vulnerabilities in LLM Agents. arXiv:2603.12614.

[64] Mahavir Dabas, Jihyun Jeong, Ming Jin. (2026). Memory-Induced Tool-Drift in LLM Agents. arXiv:2605.24941.

[65] Cheonsu Jeong, Younggun Shin. (2026). A Self-Healing Framework for Reliable LLM-Based Autonomous Agents. arXiv:2605.06737.

[66] Jiaqi Luo, Songyang Peng, Jiarun Dai. (2024). AgentGuard: An Attribute-Based Access Control Framework for Tool-Use LLM-Based Agent. arXiv:2605.28071.

[67] Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate. (2024). NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls. arXiv:2409.03797.

[68] Benjamin Elder, Anupama Murthi, Jungkoo Kang. (2025). Using Invocable APIs derived from NL2SQL datasets for LLM Tool-Calling Evaluation. arXiv:2506.11266.

[69] Zihui Wu, Haichang Gao, Jianping He. (2024). The Dark Side of Function Calling: Pathways to Jailbreaking Large Language Models. arXiv:2407.17915.

[70] Renze Lou, Baolin Peng, Wenlin Yao. (2025). The Tool Illusion: Rethinking Tool Use in Web Agents. arXiv:2604.03465.

[71] Derek Lilienthal, Sanghyun Hong. (2025). Mind the Gap: Time-of-Check to Time-of-Use Vulnerabilities in LLM-Enabled Agents. arXiv:2508.17155.

[72] Daniel Nichols, Prajwal Singhania, Charles Jekel. (2024). Optimizing Agentic Language Model Inference via Speculative Tool Calls. arXiv:2512.15834.

[73] Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos. (2024). CarbonCall: Sustainability-Aware Function Calling for Large Language Models on Edge Devices. arXiv:2504.20348.

[74] Yijuan Liang, Xinghao Chen, Yifan Ge. (2025). UniToolCall: Unifying Tool-Use Representation, Data, and Evaluation for LLM Agents. arXiv:2604.11557.

[75] Oskar Keurulainen, Gokhan Alcan, Ville Kyrki. (2024). The Role of Higher-Order Cognitive Models in Active Learning. arXiv:2401.04397.